

Overview

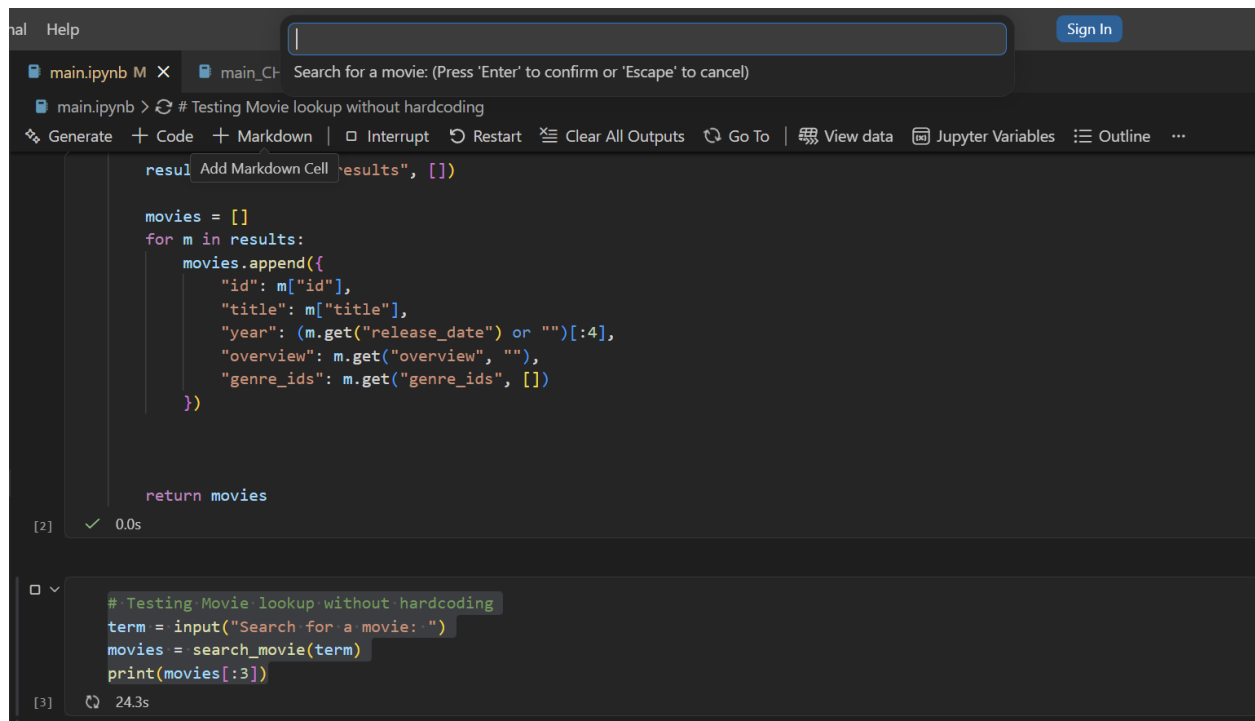
This project is a simple movie review app that uses data from the TMDB API to allow users to search for movie details, filter based on genre tags, and generate a movieboard ranking to see the top-rated films. Reviews are stored in a csv file, and genre tags are stored in a lookup table to reference when converting between genre IDs and tags.

The main goal was to create a simple review process, with a leaderboard integration to see what films are most popular.

Features Implemented

- Movie search by title using User Input.
- Submitting a review by searching and selecting a film from a list.
- Genre filtering using genres.csv
- Movieboard creation that shows top rated movies with number of reviews listed

Screenshots



The screenshot shows a Jupyter Notebook interface with a dark theme. At the top, there's a search bar with the placeholder text "Search for a movie: (Press 'Enter' to confirm or 'Escape' to cancel)". Below the search bar, the notebook has two cells. The first cell, labeled [2], contains Python code that defines a function to search for movies by title. The code iterates through results and appends movie details like id, title, year, overview, and genre_ids to a list. The second cell, labeled [3], contains code that prompts the user to enter a movie title, searches for it using the function, and prints the first three results. The interface includes standard Jupyter Notebook controls like "Generate", "Code", "Markdown", "Interrupt", "Restart", "Clear All Outputs", "Go To", "View data", "Jupyter Variables", and "Outline".

```
main.ipynb M X main_C Search for a movie: (Press 'Enter' to confirm or 'Escape' to cancel)
main.ipynb > # Testing Movie lookup without hardcoding
Generate + Code + Markdown | Interrupt Restart Clear All Outputs Go To View data Jupyter Variables Outline ...

result Add Markdown Cell results", [])

movies = []
for m in results:
    movies.append({
        "id": m["id"],
        "title": m["title"],
        "year": (m.get("release_date") or "")[:4],
        "overview": m.get("overview", ""),
        "genre_ids": m.get("genre_ids", [])
    })

return movies

[2] ✓ 0.0s

# Testing Movie lookup without hardcoding
term = input("Search for a movie: ")
movies = search_movie(term)
print(movies[:3])

[3] 24.3s
```

Users are prompted to enter a movie title.

```
# Testing Movie lookup without hardcoding
term = input("Search for a movie: ")
movies = search_movie(term)
print(movies[:3])
```

1m 23.1s Python

```
[{'id': 338969, 'title': 'The Toxic Avenger Unrated', 'year': '2025', 'overview': 'When a downtrodden janitor, Winston Gooze, is exposed to a catastrophic toxic accident,
```

Movie information is returned by the TMDb API.

```
# Test submit_review without hardcoding
term = input("Search for a movie: ")
movies = search_movie(term)

# Show search results
for i, m in enumerate(movies):
    print(f"{i+1}. {m['title']} ({m['year']})")

# User selects one from list of results
choice = int(input("Select a movie from the list using the number: "))
selected = movies[choice]

# User enters rating
rating = int(input("Enter your star review (1-5): "))

# Submit review
submit_review(selected["id"], rating, selected["genre_ids"])
```

14.1s Python

```
1. The Toxic Avenger Unrated (2025)
2. The Toxic Avenger (1984)
3. The Toxic Avenger: The Musical (2018)
4. Citizen Toxie: The Toxic Avenger IV (2001)
5. The Toxic Avenger Part II (1989)
6. The Toxic Avenger Part III: The Last Temptation of Toxie (1989)
7. Heart of Fartness: Tromm's First VR Experience Starring the Toxic Avenger (2017)
8. Dr. Platypus and the Toxic Avengers: The Probe ()
```

Same search prompt, then displayed and formatted results, before selecting one for a review.

main Help | Sign In

main.ipynb X main.Cl Select a movie from the list using the number: (Press 'Enter' to confirm or 'Escape' to cancel)

main.ipynb > # Test submit_review without hardcoding

```
...
    "genre_ids": str(genre_ids) if genre_ids is not None else "" # Add empty string if no genre id string to keep csv clean
    ...
    # Add new review to the bottom of reviews.csv
    df = pd.concat([df, pd.DataFrame([new_row])], ignore_index=True)

    # Save new review to reviews.csv
    df.to_csv("reviews.csv", index=False)
    print("Your review was saved!")
```

[4] ✓ 0.0s Python

```
# Test submit_review without hardcoding
term = input("Search for a movie: ")
movies = search_movie(term)

# Show search results
for i, m in enumerate(movies):
    print(f"{i+1}. {m['title']} ({m['year']})")

# User selects one from list of results
choice = int(input("Select a movie from the list using the number: "))
selected = movies[choice]

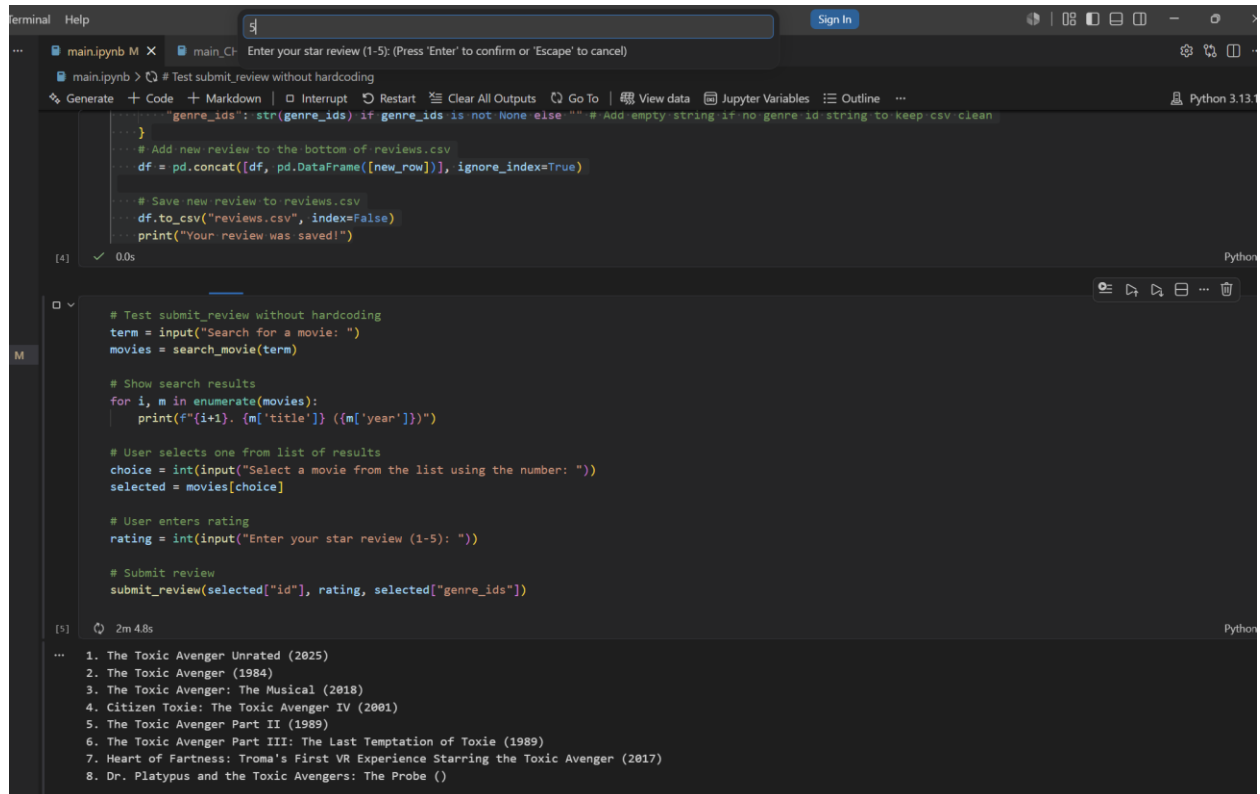
# User enters rating
rating = int(input("Enter your star review (1-5): "))

# Submit review
submit_review(selected["id"], rating, selected["genre_ids"])
```

[5] 1m 4.1s Python

```
1. The Toxic Avenger Unrated (2025)
2. The Toxic Avenger (1984)
3. The Toxic Avenger: The Musical (2018)
4. Citizen Toxie: The Toxic Avenger IV (2001)
5. The Toxic Avenger Part II (1989)
6. The Toxic Avenger Part III: The Last Temptation of Toxie (1989)
7. Heart of Fartness: Tromm's First VR Experience Starring the Toxic Avenger (2017)
8. Dr. Platypus and the Toxic Avengers: The Probe ()
```

User selects a movie to review from the list by picking a number.



The screenshot shows a Jupyter Notebook with a dark theme. The top bar includes a 'terminal' tab and a 'Help' button. The notebook has two cells. The first cell contains code for saving a review to a CSV file. The second cell contains code for searching for a movie, displaying a list of results, and submitting a review. The output of the second cell shows a list of 8 movies related to 'The Toxic Avenger'.

```
...
genre_ids": str(genre_ids) if genre_ids is not None else "" # Add empty string if no genre id string to keep csv clean
...
}
# Add new review to the bottom of reviews.csv
new_row = pd.DataFrame([new_review], ignore_index=True)
df = pd.concat([df, new_row], ignore_index=True)

# Save new review to reviews.csv
df.to_csv("reviews.csv", index=False)
print("Your review was saved!")

[4] ✓ 0.0s

# Test submit_review without hardcoding
term = input("Search for a movie: ")
movies = search_movie(term)

# Show search results
for i, m in enumerate(movies):
    print(f"{i+1}. {m['title']} ({m['year']})")

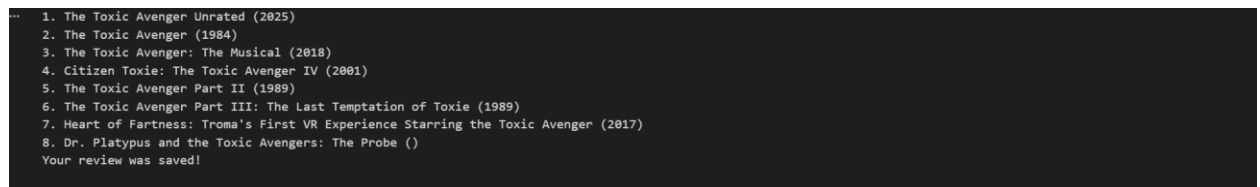
# User selects one from list of results
choice = int(input("Select a movie from the list using the number: "))
selected = movies[choice]

# User enters rating
rating = int(input("Enter your star review (1-5): "))

# Submit review
submit_review(selected["id"], rating, selected["genre_ids"])

[5] 2m4.8s

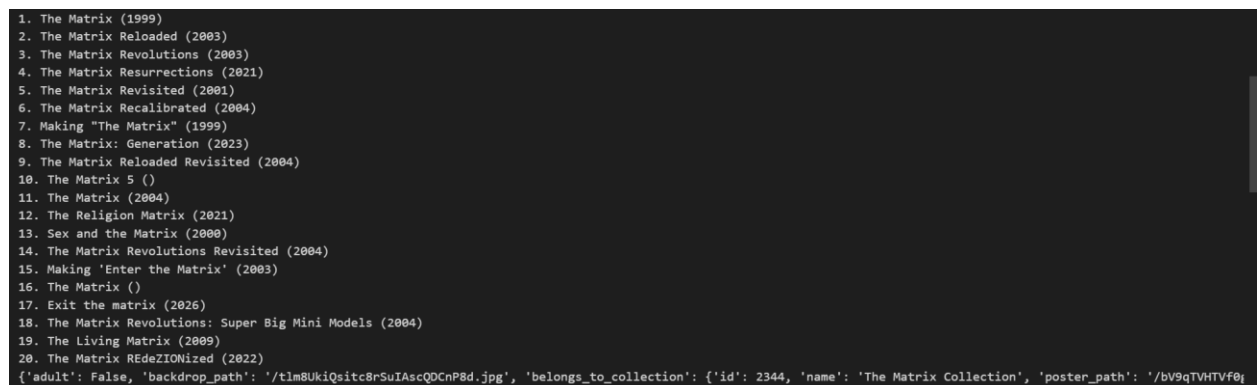
...
1. The Toxic Avenger Unrated (2025)
2. The Toxic Avenger (1984)
3. The Toxic Avenger: The Musical (2018)
4. Citizen Toxie: The Toxic Avenger IV (2001)
5. The Toxic Avenger Part II (1989)
6. The Toxic Avenger Part III: The Last Temptation of Toxie (1989)
7. Heart of Fartness: Troma's First VR Experience Starring the Toxic Avenger (2017)
8. Dr. Platypus and the Toxic Avengers: The Probe ()
```



This screenshot shows the output of the Jupyter Notebook. It displays the same list of 8 movies as the previous screenshot, followed by the message 'Your review was saved!'.

```
...
1. The Toxic Avenger Unrated (2025)
2. The Toxic Avenger (1984)
3. The Toxic Avenger: The Musical (2018)
4. Citizen Toxie: The Toxic Avenger IV (2001)
5. The Toxic Avenger Part II (1989)
6. The Toxic Avenger Part III: The Last Temptation of Toxie (1989)
7. Heart of Fartness: Troma's First VR Experience Starring the Toxic Avenger (2017)
8. Dr. Platypus and the Toxic Avengers: The Probe ()
Your review was saved!
```

After a star rating is submitted, the review is saved to the csv file.



This screenshot shows a list of 20 movie titles related to 'The Matrix'. The list includes titles like 'The Matrix (1999)', 'The Matrix Reloaded (2003)', 'The Matrix Revolutions (2003)', etc. The list is truncated at the bottom with an ellipsis.

```
1. The Matrix (1999)
2. The Matrix Reloaded (2003)
3. The Matrix Revolutions (2003)
4. The Matrix Resurrections (2021)
5. The Matrix Revisited (2001)
6. The Matrix Recalibrated (2004)
7. Making "The Matrix" (1999)
8. The Matrix: Generation (2023)
9. The Matrix Reloaded Revisited (2004)
10. The Matrix 5 ()
11. The Matrix (2004)
12. The Religion Matrix (2021)
13. Sex and the Matrix (2000)
14. The Matrix Revolutions Revisited (2004)
15. Making 'Enter the Matrix' (2003)
16. The Matrix ()
17. Exit the matrix (2026)
18. The Matrix Revolutions: Super Big Mini Models (2004)
19. The Living Matrix (2009)
20. The Matrix REdeZIONized (2022)
{'adult': False, 'backdrop_path': '/t1m8UkiQs8t8rSuIAscQDCnP8d.jpg', 'belongs_to_collection': {'id': 2344, 'name': 'The Matrix Collection', 'poster_path': '/bV9qTVHTVf0g...
```

Example of search, select and show movie details from get_movie_details().

	id	title	year	\
0	2787	Pitch Black	2000	
1	880234	Pitch Black	2021	
2	707403	Pitch Black Panacea	2020	
3	1057915	Pitch Black	2023	
4	116065	Pitch Black Heist	2012	
5	244839	Into Pitch Black	2000	
6	254631	Pitch Black: Slam City	2000	
7	1257220	Pitch-Black	1973	
8	405047	Siyaaah	2013	
9	2789	The Chronicles of Riddick	2004	

	overview	genre_ids
0	When their ship crash-lands on a remote planet...	[53, 878, 28]
1	Weary from his wife's repeated suicide attempt...	[53, 9648]
2	Amy and Carl both have lazy eyes. Amy's left a...	[16, 35]
3	Pitch Black takes us inside the claustrophobic...	[99, 18]
4	Liam and Michael are professional safe cracker...	[80, 18]
5	After the events of Pitch Black, a contractor ...	[28, 878, 80, 10770]
6	Leading up to the events of Pitch Black, Richa...	[16, 878]
7	Set in Chikuhō amid the closing of the region'...	[99]
8	Zara loses her child due to miscarriage. Bilal...	[53, 27]
9	After years of outrunning ruthless bounty hunt...	[28, 878]

Dataframe output for a search. Used to demonstrate genre filtering.

```

main Help
main.py:nb M X main.py: Enter a genre to filter by: (Press 'Enter' to confirm or 'Escape' to cancel)
main.py:nb > # Test movies_with_genre() filtering
Generate + Code + Markdown | Interrupt Restart Clear All Outputs Go To View data Jupyter Variables Outline Python 3.13.1
[9] 1m 0.4s
...
    id          title  year  \
0    2787    Pitch Black  2000
1   880234    Pitch Black  2021
2   707403  Pitch Black Panacea  2020
3   1057915    Pitch Black  2023
4   116065    Pitch Black Heist  2012
5   244839    Into Pitch Black  2000
6   254631  Pitch Black: Slam City  2000
7   1257220    Pitch-Black  1973
8   405047      Siyaah  2013
9    2789  The Chronicles of Riddick  2004

    overview          genre_ids
0  When their ship crash-lands on a remote planet...  [53, 878, 28]
1  Weary from his wife's repeated suicide attempt...  [53, 9648]
2  Amy and Carl both have lazy eyes. Amy's left a...  [16, 35]
3  Pitch Black takes us inside the claustrophobic...  [99, 18]
4  Liam and Michael are professional safe cracker...  [80, 18]
5  After the events of Pitch Black, a contractor ...  [28, 878, 80, 10770]
6  Leading up to the events of Pitch Black, Richa...  [16, 878]
7  Set in Chikuhō amid the closing of the region'...  [99]
8  Zara loses her child due to miscarriage. Bilal...  [53, 27]
9  After years of outrunning ruthless bounty hunt...  [28, 878]

```

User selects a genre to filter by.

```

...      id          title  year \
0      2787          Pitch Black  2000
1      880234          Pitch Black  2021
2      707403      Pitch Black Panacea  2020
3      1057915          Pitch Black  2023
4      116065          Pitch Black Heist  2012
5      244839      Into Pitch Black  2000
6      254631      Pitch Black: Slam City  2000
7      1257220          Pitch-Black  1973
8      405047          Siyaah  2013
9      2789      The Chronicles of Riddick  2004

      overview      genre_ids
0  When their ship crash-lands on a remote planet...  [53, 878, 28]
1  Weary from his wife's repeated suicide attempt...  [53, 9648]
2  Amy and Carl both have lazy eyes. Amy's left a...  [16, 35]
3  Pitch Black takes us inside the claustrophobic...  [99, 18]
4  Liam and Michael are professional safe cracker...  [80, 18]
5  After the events of Pitch Black, a contractor ...  [28, 878, 80, 10770]
6  Leading up to the events of Pitch Black, Richa...  [16, 878]
7  Set in Chikuko amid the closing of the region'...  [99]
8  Zara loses her child due to miscarriage. Bilal...  [53, 27]
9  After years of outrunning ruthless bounty hunt...  [28, 878]

      id  title  year      overview \
8  405047  Siyaah  2013  Zara loses her child due to miscarriage. Bilal...

      genre_ids
8  [53, 27]

```

Results are filtered by user selected genre.

```

# Test generate_movieboard
generate_movieboard()
✓ 19s Python

Top movies:
Idiocracy (Comedy, Science Fiction, Adventure) -- Avg: 5.00 from 1 reviews
The Toxic Avenger: The Musical (Comedy, Horror, Science Fiction, Music) -- Avg: 5.00 from 1 reviews
Star Wars Dreams (TV Movie, Documentary) -- Avg: 5.00 from 1 reviews
Interstellar (Adventure, Drama, Science Fiction) -- Avg: 3.00 from 1 reviews

```

Example movieboard output, with genre tags.

Issues Encountered

1. NoneType error while testing. TMDB was not returning the expected information. The printouts were added to check if the correct information was being received. The issue was fixed after a few retries of running the same search test.
2. The correct number of ratings were not displayed after using `generate_movieboard()`. There was NaN and 0 being displayed as placeholders. Ratings and reviews were not in the correct column order. The system ended up creating another column that confused the order. I made the standard term for the 1-5 star system to be rating and updated the columns to be the correct order. These changes fixed the issue.
3. Key error while implementing genre filtering. Column order was not correct. Genre ID was supposed to be listed first, to convert to the word. I updated genres.csv with the correct order of information and the issue was resolved.

Other than these issues, I have yet to encounter a bug that I couldn't fix after some deliberation.

Milestones

M1: Add a menu system UI for users to interact with. Finish by end of day 7/13.

Users would be able to select options from a menu to either search for a movie, review a movie, filter by genre, and see the movieboard. With implementation of the next milestone, users would also be able to receive a recommendation based on their review history and genre preferences.

M2: Implement a recommendation system. Finish by end of day 7/20.

Users would be able to receive a recommendation from the system based on their review history and genre preferences. Data from genres.csv and reviews.csv will be used to determine recommendations.

M3: Input Validation, Error Handling and Cleanup. Finish by end of day 7/27.

Ensure user input validation and error handling are sufficient. Clean up code and edit and add documentation where needed.

Self Reflection

I think I have made pretty good progress so far, considering I am juggling two projects between my courses currently. I believe if I keep track of each milestone, I can get this project completed within the next few weeks. I found it easy to lay out each function and understand their purposes and what I was trying to receive for output, but what was more difficult was understanding the processing level to trace back errors if I got them. I think solving bugs though has been the "light bulb" moments so to say, when they are finally solved. After having a couple similar bugs connected to csv column order, I now recognize that I need to pay attention to the order I have information in so the system can read it correctly. Moving the recommendation system to Version 2 also helps give a realistic timeline to implement that feature. I believe in the next few weeks, I can flesh this out to look more like a full application rather than the current skeleton.